University of Hertfordshire

**School of Engineering and Computer Science**

**MSc Computer Science**

**Artificial Intelligence and Robotics**

Module: MSc CS Project

# Interim Progress Report

# FPGA-Based Driver Distraction Detection System Using Image Classification



| | | |
|---|---|---|
| Name | : | Dulhara Anjana Langappuli |
| Student ID | : | 22044421 |
| Level | : | Level 7 |
| Academic Year | : | 2025-26 |
| Supervisor | : | Professor Saeid Gorgin |

# Contents

# List of Figures



# List of Tables

## Executive Summary

This Interim Progress Report presents the current state of the MSc project which aims at driver distraction recognition through image classification and FPGA implementation. Tasks performed to date include data set validation, data partitioning per subject, implementation of three Convolutional Neural Networks (CNN), their comparative analysis and domain shift exploration.

State Farm Distracted Driver Detection data set contains 22,424 images belonging to 26 subjects. Data partitioning has been carried out by subject rather than by images. This resulted in 15,899 images for training set, 3,439 images for validation and 3,086 images for test sets. There is no subject overlap between these three sets. This approach allows us to perform more strict evaluation than image random splitting because all models are tested on drivers who have not appeared in training phase before.

Three image-classification models have been analysed with the use of the same subject-based protocol. Baseline CNN model reached 75.60% test accuracy. EfficientNet-B0 model demonstrated 82.47% test accuracy, whereas MobileNetV2 proved to be the best model with 84.67% test accuracy, 84.01% macro F1-score and 84.94% weighted F1-score. Moreover, MobileNetV2 model has reached approximately 16.64 Frames per Second (FPS) on the development laptop, in contrast to 17.61 FPS for baseline CNN and 15.50 FPS for EfficientNet-B0.

A qualitative test based on images taken outside of vehicle environment has shown that all models yield inconsistent results and are very sensitive to the changes in background, camera view point, driver posture and image capturing conditions. Hence, next tasks to be fulfilled include robustness test under low-light and IR-like synthetic environment, in-car evaluation, model quantization, FPGA implementation and electronic alert system integration.

## 1.0 Introduction

### 1.1 Problem and motivation

Driver distraction is a road safety problem whereby attention is distracted from driving due to such activities as the use of a mobile phone, texting, drinking, fiddling with vehicle controls, reaching behind the driver, grooming, or passenger interaction.

The chosen dataset consists of ten driver actions, some of these differ mainly through slight differences in hand positioning, orientation and body posture. A practical system needs to detect these visual differences while remaining invariant to changes in the driver's appearance, clothes, vehicle interior, camera position and lighting conditions.

In the original proposal, there was a plan for developing a more general driver drowsiness and alerting system. As a result of the discussion with the supervisor, the project scope was reduced to driver distraction classification. In addition to other benefits, this modification led to a labelled publicly available dataset and clear experiment setup with realistic opportunities for implementation on embedded systems.

FPGA module was retained in the system as it adds hardware limitations which cannot be measured using only classification performance. The intended result is therefore not limited to a trained classification model. The final artefact is to include the classification pipeline with robustness analysis, FPGA module and electronic alert generator.

### 1.2 Aim and objectives

**Aim:** To design and evaluate an FPGA-based driver distraction detection prototype that uses image data to classify driver behaviour and provides an alert when distracted behaviour is identified.

The project has two principal objectives:

1. To prepare and use a suitable driver-distraction image dataset for training, testing and evaluating a driver-behaviour classification system.
2. To develop and evaluate a driver-distraction detection prototype using FPGA-related hardware design and electronic alert outputs.

### 1.3 Research questions

1. How do a custom CNN, MobileNetV2 and EfficientNet-B0 compare when the test drivers are not present in the training data?
2. Which model provides the strongest balance between classification performance and inference throughput for later edge or FPGA-oriented work?
3. How sensitive is the selected model to domain shift and adverse image conditions, particularly low light and synthetic IR-like transformations?
4. Which component of the final pipeline can be implemented or simulated credibly using FPGA tools within the remaining project period?

## 2.0 Background Research and Literature Review

### 2.1 Image-based driver monitoring

Driver monitoring research uses visual, vehicular and physiological data. According to Kashevnik et al. (2021) the methodologies available may be classified based on driver state, vehicle state and environmental variables. The method proposed by the authors includes an inside camera for use in this study since the state of manual and visual distraction can be determined through the analysis of driver hand placement, body position and head orientation. From their literature review, it is clear that besides the accuracy of recognition, conditions of sensing, processing delay and the nature of warnings are key components of an effective system.

The results obtained through image-based deep learning have been remarkable. Valeriano et al. (2018) showed that deep convolutional neural networks are capable of detecting distracted driving behaviours. In addition, according to AlShalfan and Zakariah (2021) 96.95% accuracy was achieved using the VGG-16 model on State Farm dataset. Although these results confirm the learnability of the dataset, comparability of the results will be complicated in the case where the partitioning process is not specified. Repeated images of the same driver might contain similar background, clothes and even camera hints.

### 2.2 Subject-wise evaluation and domain shift

Aljasim and Kashef (2022) divided the validation and test sets using driver identifiers. Their individual models achieved results in the low-to-high 80% range, while the ensemble produced a higher result. These results are more comparable with the present work than results obtained using an unspecified random image split. A lower score under a stricter cross-driver protocol can still provide more meaningful evidence because the model is required to generalise to unseen drivers.

generalisation becomes even harder in case the capture setting changes too. The cross-dataset and cross-driver transfer was studied in Wang and Wu (2023) through domain adaptation. The results prove the conclusion made after the qualitative testing: success in an unseen portion of State Farm dataset does not mean good performance in a new car and a new background/camera configuration.

### 2.3 Architecture selection and practical application

MobileNetV2 reduces the computational cost using inverted residual block, linear bottleneck and depthwise separable convolution (Sandler et al., 2018). On the other hand, the EfficientNet-B0 applies the compound scaling to achieve a balance between depth, width and input resolution (Tan and Le, 2019). Both architectures offer a reasonable efficiency benchmark for a project that will continue with the hardware-oriented development. A custom CNN was chosen as a baseline to measure the transfer learning effect instead of assuming it

In the experimental process, the literature had four practical implications. First, the data were split based on driver identifiers. Second, horizontal flips were not applied, as it would change a right-side movement into a left-side one with the same label. Third, all models worked with $224 \times 224$ input size to allow for comparison. Fourth, evaluation included macro and weighted F1-score, confusion matrix, latency and frames per second (FPS) apart from accuracy.

## 3.0 Summary of Progress to Date

### 3.1 Dataset audit and preparation

State Farm Distracted Driver Detection dataset was audited before the creation of the model (State Farm, 2016). Metadata contains 22,424 labelledimages belonging to 26 drivers. All 22,424 paths to labelledimages were contained in the local copy of the dataset and the resolution of the original image was 640 × 480.

The folder of the Kaggle test competition does not contain public class labels. As a result, labelledtraining data were divided into training, validation and testing parts according to the needs of the particular project.

The dataset consists of ten classes of driver behaviours. Safe driving is the largest class with 2,489 images and hair and makeup is the smallest one with 1,911 images. While the class distribution is not entirely even, the difference between the largest and the smallest class is not great. Therefore, accuracy score along with macro and weighted F1-score was used for the evaluation.


**Figure 1:** Examples from the ten driver-behaviour classes in the State Farm dataset.

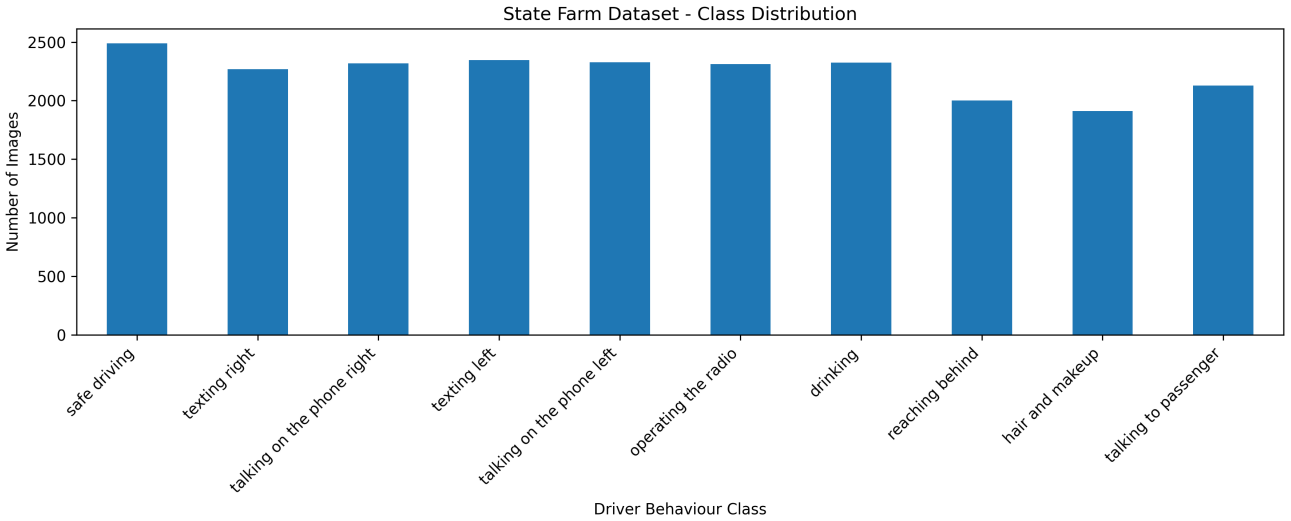



**Figure 2:** Class distribution of the labelled State Farm dataset.

**Table 1:** State Farm class distribution used in the project.

| Class | Behaviour | Images |
|---|---|---|
| c0 | Safe driving | 2,489 |
| c1 | Texting - right | 2,267 |
| c2 | Phone use - right | 2,317 |
| c3 | Texting - left | 2,346 |
| c4 | Phone use - left | 2,326 |
| c5 | Operating the radio | 2,312 |
| c6 | Drinking | 2,325 |
| c7 | Reaching behind | 2,002 |
| c8 | Hair and make-up | 1,911 |
| c9 | Talking to a passenger | 2,129 |

### 3.2 Subject-wise split

The datasets have been split using the subject ID obtained from "driver_imgs_list.csv". As a result, the training dataset consists of 15,899 images from 18 drivers while the validation and test datasets consist of 3,439 and 3,086 images from 4 drivers each.

For testing purposes, p002, p012, p045 and p075 subjects were used. None of these drivers are present in more than one set. Lack of any overlap among subjects was validated via programmatic means prior to training the models.

This methodology was adopted to avoid subject leakage. Subject leakage means that during a random image split, there can be a situation where images of the same driver will end up in both the training and the test sets, thereby allowing the model to make use of repeating clothes, body parts, interior of the car and camera placement instead of learning behavioural features for a new driver.

The split data were saved into CSV files which were then used in all three experiments to evaluate the performance of the architectures on identical datasets.

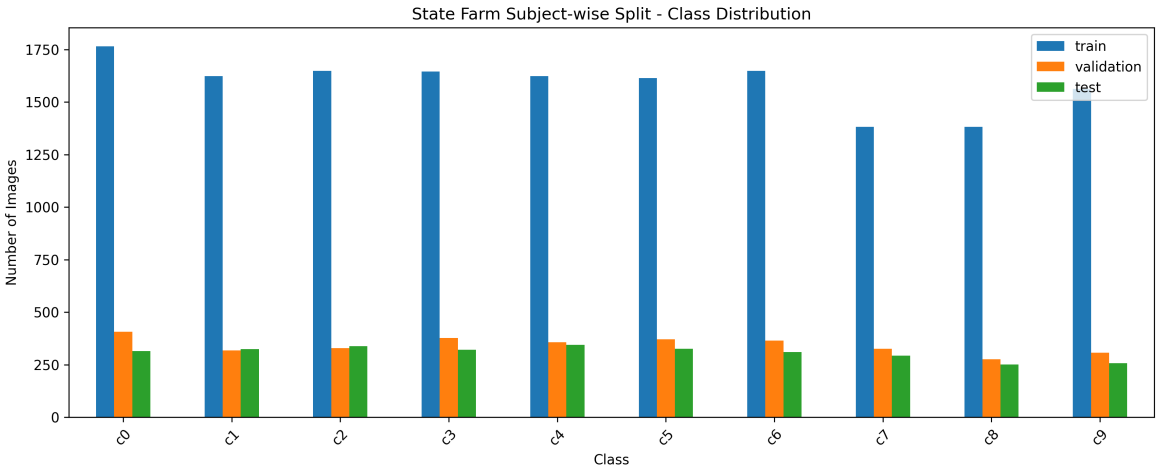



**Figure 3:** Class counts in the subject-wise training, validation and test partitions.

### 3.3 Development environment and reproducibility

The computational environment was set up using Python 3.10, TensorFlow 2.10.1, CUDA 11.2, cuDNN 8.1, NumPy 1.23.5 and OpenCV 4.8.1.78.

During initial TensorFlow setup, the NVIDIA RTX 3060 Laptop GPU could not be recognisedin native Windows. Compatibility requirements check showed that the TensorFlow release installed was not compatible with the expected GPU setup in native Windows. As a result, a separate Anaconda environment was created using TensorFlow 2.10.1 along with appropriate versions of CUDA, cuDNN, NumPy and OpenCV.

The setup successfully recognisedthe NVIDIA GPU and was identified as a separate Jupyter kernel. Furthermore, a launcher was made for launching the correct environment and JupyterLab from the project folder.

This setup helps to avoid running experiments in the wrong Python environment and using TensorFlow setup without GPU support.

The repository contains notebooks, split files, experiment output data, figures, tables, documents and reusable source code modules. Image datasets, independently collected test images and model checkpoints were ignored through ".gitignore". This way, version control focuses on reproducible code, configuration, results and documentation without accidental dataset distribution.

### 3.4 Model implementation and technical issues

A custom-designed convolutional neural network served as the baseline model. It consists of convolutional layers, batch normalisation, max pooling, global average pooling and dropout and totals up to 424,522 parameters. Data augmentation included small rotations, zooms and contrasts.

The horizontal flipping was not included as the dataset contained separate classes for behaviours occurring on the right side and on the left side of a person. Applying the horizontal flipping could make one image look like a left side behaviour while keeping the label the same, resulting in training data with errors.

It was discovered that there is a discrepancy between the checkpoint used for early stopping and the checkpoint saved as the best result in the baseline experiment. While the former relied on validation loss, the latter used validation accuracy. Therefore, the checkpoint with the best accuracy was manually loaded and tested to ensure reproducibility of the results. The results provided in the report relate to this checkpoint.

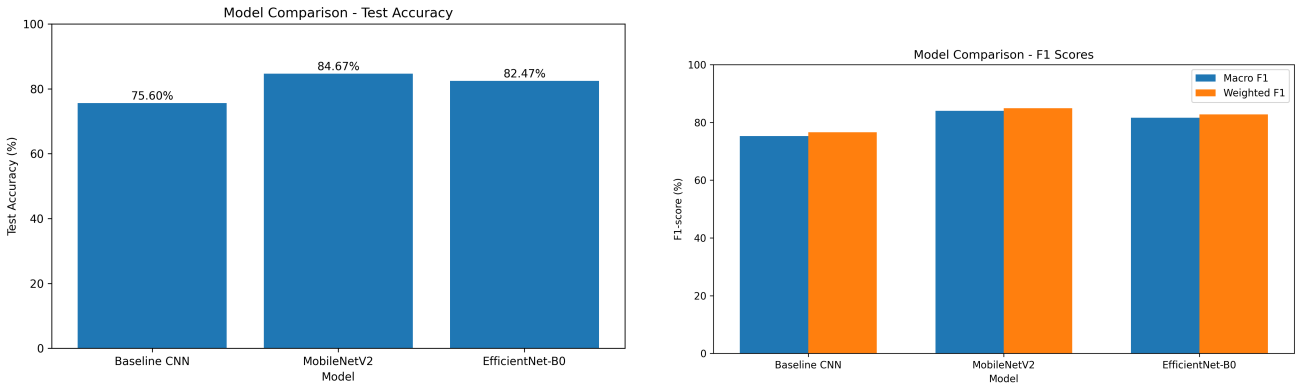
MobileNetV2 and EfficientNet-B0 models were fine-tuned using a two-step transfer learning approach. First, the pretrained feature extraction portion was frozen and only the newly created classification portion was trained. Then, selected layers from the pretrained model were unfrozen and fine-tuned using a smaller learning rate.

EfficientNet-B0 yielded a TensorFlow/Keras error when trying to save the whole model. To resolve this, weights-only checkpoints were used. Later, a weight loading mismatch problem appeared while trying to reconstruct the model in another notebook. The original outputs and checkpoint files were kept. The last additional step was required to reproduce the result – the selected model had to be exported and loaded correctly in a separate notebook.

### 3.5 Comparative results

**Table 2:** Model performance under the common subject-wise protocol.

| Model | Val. accuracy | Test accuracy | Macro F1 | Weighted F1 | Training time |
|---|---|---|---|---|---|
| Baseline CNN | 60.89% | 75.60% | 75.27% | 76.57% | 11.87 min |
| MobileNetV2 | 82.12% | 84.67% | 84.01% | 84.94% | 32.03 min |
| EfficientNet-B0 | 72.23% | 82.47% | 81.58% | 82.79% | 37.57 min |




**(a)** Test accuracy comparison.

**(b)** Macro and weighted F1 comparison.

**Figure 4:** Comparison of the three implemented models.

MobileNetV2 produced the strongest overall classification performance. Its test accuracy of 84.67% was 9.07 percentage points higher than the custom baseline CNN and 2.20 percentage points higher than EfficientNet-B0.

The same ordering was observed for the macro and weighted F1-scores. This indicates that the improvement was not limited to the most frequent classes. MobileNetV2 achieved a macro F1-score of 84.01% and a weighted F1-score of 84.94%.

The baseline CNN produced the highest measured throughput at 17.61 FPS, but its lower classification performance reduced its suitability for further development. MobileNetV2 achieved 16.64 FPS, representing a reduction of less than one frame per second compared with the baseline while producing a substantial improvement in test accuracy.

EfficientNet-B0 achieved 82.47% test accuracy and 15.50 FPS. It required more training time than MobileNetV2 and introduced additional model-export difficulties. MobileNetV2 was therefore selected as the most suitable candidate for the next stage of robustness, quantisation and FPGA-oriented investigation.

**Table 3:** Deployment-oriented measurements on the development laptop.

| Model | Model size | Average latency | FPS | Measurement note |
|---|---|---|---|---|
| Baseline CNN | 4.96 MB | 56.79 ms | 17.61 | Trained model loaded. |
| MobileNetV2 | 20.78 MB | 60.08 ms | 16.64 | Trained model loaded. |
| EfficientNet-B0 | 15.75 MB | 64.51 ms | 15.50 | Architecture speed measured; trained accuracy taken from saved evaluation. |

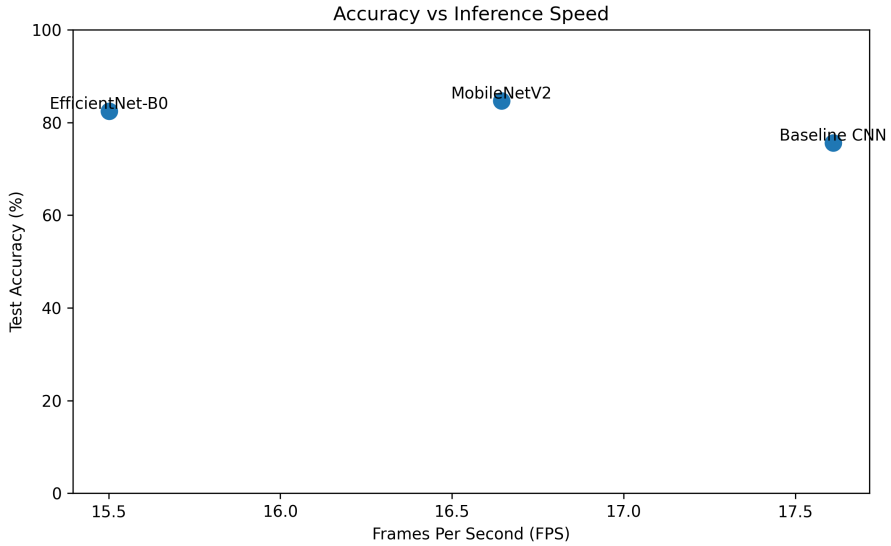



**Figure 5:** Accuracy-throughput trade-off for the three models.

### 3.6 Confusion matrix and qualitative domain-shift test

As for the class-level evaluation of MobileNetV2, this model showed good results on some well-distinguished manual activities. The most impressive F1-scores were obtained for texting left, phone use right, radio activity and reaching behind.

Poor results were found for talking to a passenger, safe driving and hair and make-up. The classes are more varied in terms of posture and some of them share visual features with neighboring actions. Safe driving photos can have some movements with hands and heads similar to distracted behaviour and talking to a passenger can be mostly characterized by head direction and context.

Also, a short qualitative analysis was performed on one State Farm photo and three independently taken photos illustrating drinking and phone use. All three models recognisedthe State Farm photo. Independent photos were classified differently.

EfficientNet-B0 correctly recognisedone drinking photo and left-handed phone usage. MobileNetV2 classified the second drinking photo. The baseline CNN suggested operating the radio action in all three independent photos.

Independent photos were captured in the university library rather than inside the car. The background had no steering wheel and dashboard, the camera view was different, the light conditions were also far away from

those used in the training set. Thus, the test is not considered to be in-car accuracy check.

This result is interpreted as an example of domain shift, which means that the models have learned context-specific information which does not work in a new environment. This fact proves the necessity of adding controlled low-light testing and synthetic IR-like transformation to the remaining plan.

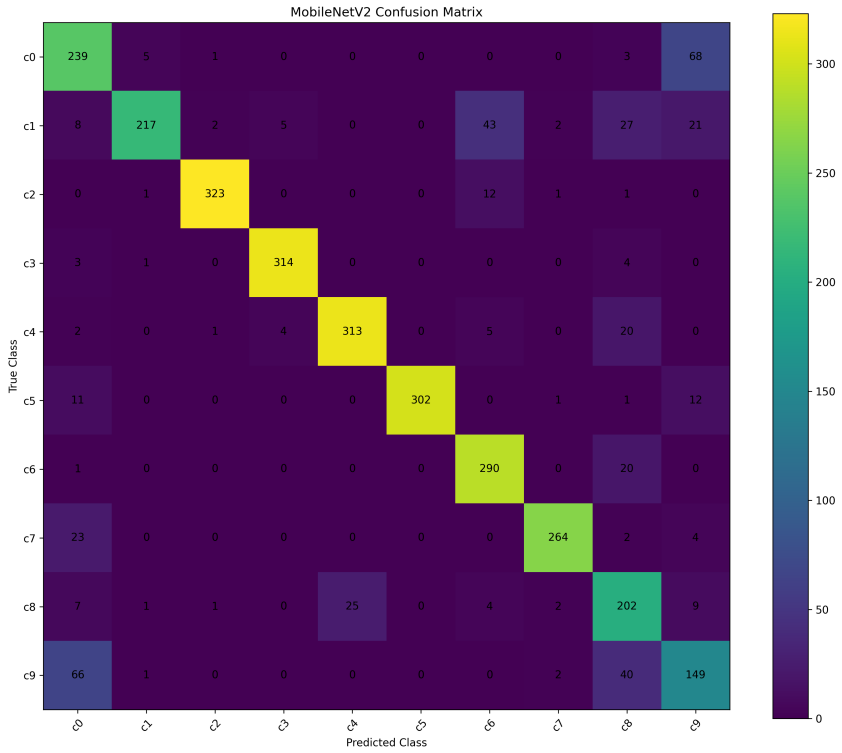



**Figure 6:** Confusion matrix for the selected MobileNetV2 model.

### 3.7 Deliverables completed

The following deliverables were completed or substantially completed at the IPR stage:

- Detailed Project Proposal and initial project timeline;
- PRISMA-informed literature survey (Page et al., 2021);
- structured research-paper collection and literature-analysis figures;
- dataset integrity audit and class-distribution analysis;
- driver-subject distribution analysis;
- reproducible subject-wise training, validation and test partitions;
- custom baseline CNN implementation and evaluation;
- MobileNetV2 training and fine-tuning;
- EfficientNet-B0 training and fine-tuning;
- classification reports and confusion matrices;
- comparative accuracy and F1-score analysis;
- model-size, latency and FPS evaluation;
- qualitative domain-shift analysis;
- structured private GitHub repository; and
- updated robustness, quantisation and FPGA implementation plan.

## 4.0 Ethical, Legal, Professional and Social Issues

### 4.1 Ethics and research procedure

Ethics approval is not currently required because the project uses an existing public secondary dataset and does not involve participant recruitment or new data collection. All training and evaluation procedures carried out up to now use only the publicly available State Farm Distracted Driver Detection dataset, provided by Kaggle.

This dataset contains annotated driver images and subject information and is utilized exclusively for building and testing classification models. New images of other people are not collected in this project. Files containing the dataset are stored locally and are not included in the GitHub repository.

### 4.2 Privacy, fairness and safety

Driver facing images from the publicly available State Farm dataset are used exclusively for training, validating and testing of behaviour-classification models. This dataset is stored locally and is not uploaded into the GitHub repository.

The dataset does not contain any information about drivers like their age, skin color, physique and other personal details. Thus, this project cannot claim that the model will have the same performance for all drivers and in all vehicle conditions. Subject-wise testing evaluates the performance on unknown drivers, but does not remove all possible biases. Therefore, the final evaluation will include class-level errors and will explicitly point at the limitations of the dataset.

Also, the system can make wrong predictions. Missing distraction might not trigger an alert, while false alerts might distract and irritate the driver. Therefore, the created system will be considered a research prototype and not a ready-to-use safety product and further testing is necessary for its implementation.

### 4.3 Professional conduct and intellectual property

Key features of the BCS Code of Conduct include privacy, security, professional competence and the proper communication of findings (BCS, The Chartered Institute for IT, 2026). Those issues relate directly to the current project as there will be an explanation of the limitations and difficulties involved.

The State Farm data set does not go into the project repository and large files of the trained model are kept apart. References to academic papers, datasets, pretrained models and external approaches used are given through Harvard style citations.

**Table 4:** Main controls for the remaining project work.

| Issue | Control |
|---|---|
| Use of driver images | Use only the publicly available State Farm dataset, store the files locally and exclude them from the GitHub repository. |
| Privacy | Store local images securely, exclude them from GitHub and minimise retention. |
| Weak generalisation or bias | Retain subject-wise testing, inspect class-level errors and state population limitations. |
| Unsafe interpretation | Label the system as a research prototype and separate recognition results from safety validation. |
| Reproducibility | Preserve environment details, split files, logs and export-and-reload tests. |

## 5.0 Project Management and Remaining Plan

### 5.1 Management approach

Project management is based on the deadlines of the university and its DPP, IPR and final report. The technical aspect includes separate steps that answer the research questions or bring improvements to the work done.

Results, any changes in the code and key decisions are registered by means of Git, project notes and supervisor meetings. This project is based on the two main goals, while other model experiments will be taken only if there is enough time. Evidence of the repository structure, version-control activity and supervisor meeting records is provided in Appendix C.

### 5.2 Remaining tasks and deliverables

**Table 5:** Remaining tasks and expected evidence.

| Task | Method and deliverable |
|---|---|
| Low-light and IR-like robustness | Apply brightness, contrast, grayscale, noise, blur and clearly labelled synthetic IR-like transformations to the fixed test set. Produce a robustness table and class-level analysis. |
| Additional robustness evaluation | Evaluate the selected model using controlled image transformations and, where suitable, another publicly available labelled dataset. No new participant images will be collected. |
| Quantisation and deployment analysis | Prepare MobileNetV2 for TensorFlow Lite, ONNX or a suitable FPGA tool flow; compare accuracy, size and latency before and after quantisation. |
| FPGA-supported component | Implement or simulate a defensible component such as preprocessing, fixed-point support, decision logic or alert control. Preserve simulation, timing and resource evidence. |
| Alert integration | Connect the safe/distracted decision to a buzzer, LED or visual indicator. |
| Final validation and report | Repeat key tests, verify saved artefacts, consolidate results and complete the report and demonstration. |

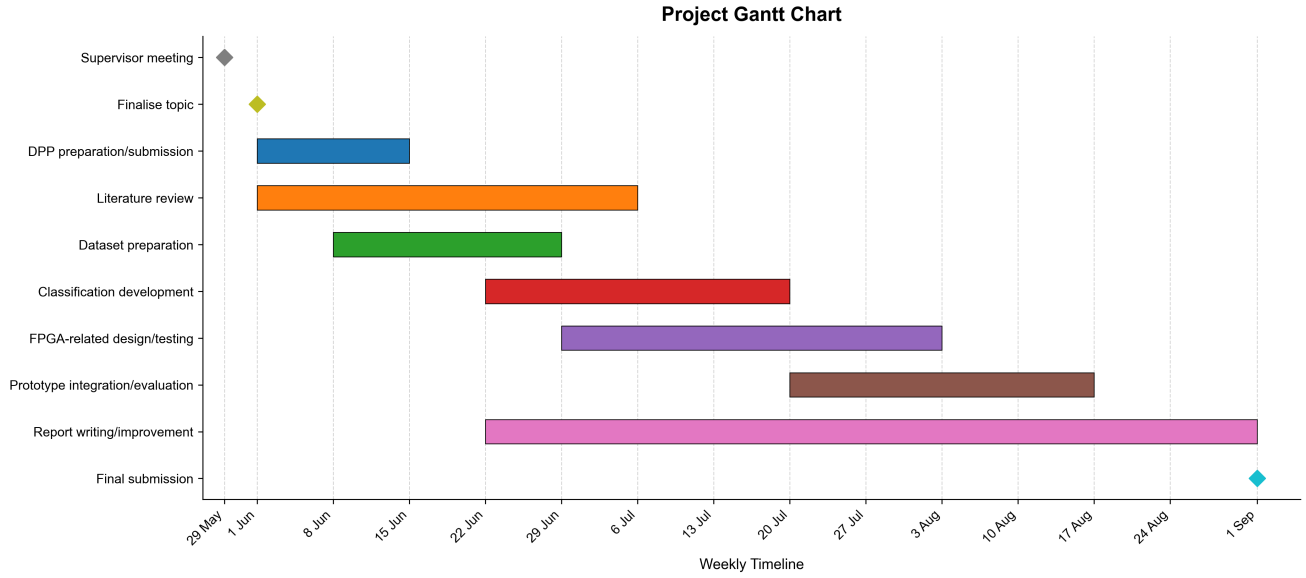



**Figure 7:** Project timeline showing the planned activities from May to September 2026.

### 5.3 Evaluation and contingency

The test-set, in its current subject-wise form, will form the basis for the comparison dataset. The final testing will give the performance metrics like accuracy, precision, recall, macro F1, weighted F1, confusion matrices, latency and frames per second (FPS). The robustness results will be compared to those from the original RGB implementation. The quantised/hardware-supported implementations will be tested for any differences in accuracy, latency, model size and any Look-Up Tables (LUT), Digital Signal Processing (DSP), Block RAM (BRAM) resources used.

The main danger will be that complete implementation of CNN on FPGA cannot be done due to time or toolchain constraints. The contingency plan will be to implement a smaller but supported FPGA component like fixed-point preprocessing block, decision controller, or alert interface with simulation, timing and resource analysis provided. This approach ensures that there is hardware implementation done without claiming that something was implemented that could not be shown.

## 6.0 MSc-Level Nature of the Project and Critical Reflection

### 6.1 Level of the artefact and investigation

The artefact goes beyond a notebook which produces a classifier. It combines the controlled dataset protocol, three different comparative neural network architectures, the deployment results, robustness testing and FPGA-specific implementation route. In the final demonstrator, the classification decision will be integrated with the alert and will describe the processing location, introduced delay and any limitations.

This work follows an experimental protocol. The custom CNN gives the initial baseline. MobileNetV2 and EfficientNet-B0 will check whether the transfer learning improves the performance of the model on unknown drivers. The subject-wise splitting will check how the model can generalize on unknown subjects. F1-scores and confusion matrices will measure the behaviour of classes, while the latency and FPS impose the deployment constraints. Finally, the qualitative test will modify the visual domain and expose a weakness in the benchmarking result.

### 6.2 Reflection on methods and limitations

The comparison of the three models showed that the way the dataset is divided has an important effect on the final results. The subject-wise split was more difficult than a random image split because the models were tested using drivers that were not included during training. However, this gave a more realistic indication of how the models may perform with unseen drivers.

The small domain-shift test also showed that good results on the State Farm dataset do not always transfer to images taken in a different environment. Changes in the background, camera position, lighting and driver posture affected the predictions.

Some technical problems were also identified during model development. In the baseline experiment, the early-stopping condition and model-checkpoint condition were based on different validation measurements. This was corrected by reloading and evaluating the saved best model. EfficientNet-B0 also produced problems when saving and loading the complete model. This showed that model saving and reloading must be tested before the final model can be considered reproducible.

There are still several limitations. The project currently uses only one public dataset, so the results cannot represent every real driving situation. The measured FPS values were obtained using a laptop and cannot be treated as FPGA performance. The current models also classify individual images and do not use information from a sequence of video frames. In addition, synthetic IR-like images cannot fully represent images captured using a real infrared camera.

The FPGA-supported component and the electronic alert output have not yet been completed. These areas will be addressed during the remaining project work and will be discussed clearly in the final report.

### 6.3 Professional relevance

The project brings together machine learning, computer vision, embedded systems, electronics and FPGA-related hardware design. These areas are relevant to the MSc programme and to the development of practical intelligent systems.

The work completed so far has provided experience in setting up the software environment, training models using a GPU, applying transfer learning, comparing model performance and managing the project code using Git. It has also improved understanding of subject-wise evaluation, model limitations and technical documentation.

The remaining work on model quantisation, FPGA implementation and alert integration will move the project from software-based model comparison towards a working hardware-supported prototype.

## 7.0 Conclusion

The project has made good progress at the interim stage. The dataset has been checked and prepared, the software environment has been configured and a subject-wise split has been used for training, validation and testing. Three image-classification models have also been trained and evaluated using the same dataset partitions.

MobileNetV2 produced the best overall result, with 84.67% test accuracy, 84.01% macro F1-score and 84.94% weighted F1-score. It also achieved approximately 16.64 FPS on the development laptop. These results made MobileNetV2 the current choice for the next stage of the project.

The domain-shift test showed that good performance on the State Farm dataset does not guarantee the same performance in a different environment. Changes in background, camera position, lighting and driver posture affected the model predictions.

The first project objective is mostly complete because the dataset has been prepared and the three selected

models have been trained and compared. The second objective is still in progress because the FPGA-supported component and electronic alert output have not yet been completed.

The remaining work will focus on robustness testing, additional robustness evaluation using existing or publicly available data, model quantisation, FPGA-related implementation, alert integration and final validation. The results from these stages will be used to evaluate whether the selected model can be included in a practical driver-distraction detection prototype.

## References


1. Aljasim, M. and Kashef, R. (2022). 'E2DR: A Deep Learning Ensemble-Based Driver Distraction Detection with Recommendations Model'. *Sensors* 22.5, p. 1858. DOI: 10.3390/s22051858.
2. AlShalfan, K. A. and Zakariah, M. (2021). 'Detecting Driver Distraction Using Deep-Learning Approach'. *Computers, Materials & Continua* 68.1, pp. 689–704. DOI: 10.32604/cmc.2021.015989.
3. BCS, The Chartered Institute for IT (2026). *BCS Code of Conduct*. Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/ (Accessed: 17th July 2026).
4. Kashevnik, A., Shchedrin, R., Kaiser, C. and Stocker, A. (2021). 'Driver Distraction Detection Methods: A Literature Review and Framework'. *IEEE Access* 9, pp. 60063–60076. DOI: 10.1109/ACCESS.2021.3073599.
5. Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hrobjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., McGuinness, L. A., Stewart, L. A., Thomas, J., Tricco, A. C., Welch, V. A., Whiting, P. and Moher, D. (2021). 'The PRISMA 2020 Statement: An Updated Guideline for Reporting Systematic Reviews'. *BMJ* 372, n71. DOI: 10.1136/bmj.n71.
6. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A. and Chen, L.-C. (2018). 'MobileNetV2: Inverted Residuals and Linear Bottlenecks'. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4510–4520. DOI: 10.1109/CVPR.2018.00474.
7. State Farm (2016). *State Farm Distracted Driver Detection. Kaggle Competition Dataset*. Available at: https://www.kaggle.com/competitions/state-farm-distracted-driver-detection (Accessed: 17th July 2026).
8. Tan, M. and Le, Q. V. (2019). 'EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks'. In: *Proceedings of the 36th International Conference on Machine Learning*. Vol. 97. Proceedings of Machine Learning Research, pp. 6105–6114.
9. Valeriano, L. C., Napoletano, P. and Schettini, R. (2018). 'Recognition of Driver Distractions Using Deep Learning'. In: *2018 IEEE 8th International Conference on Consumer Electronics – Berlin*. DOI: 10.1109/ICCE-Berlin.2018.8576183.
10. Wang, J. and Wu, Z. (2023). 'Driver Distraction Detection via Multi-Scale Domain Adaptation Network'. *IET Intelligent Transport Systems*. DOI: 10.1049/itr2.12366.

# Appendices

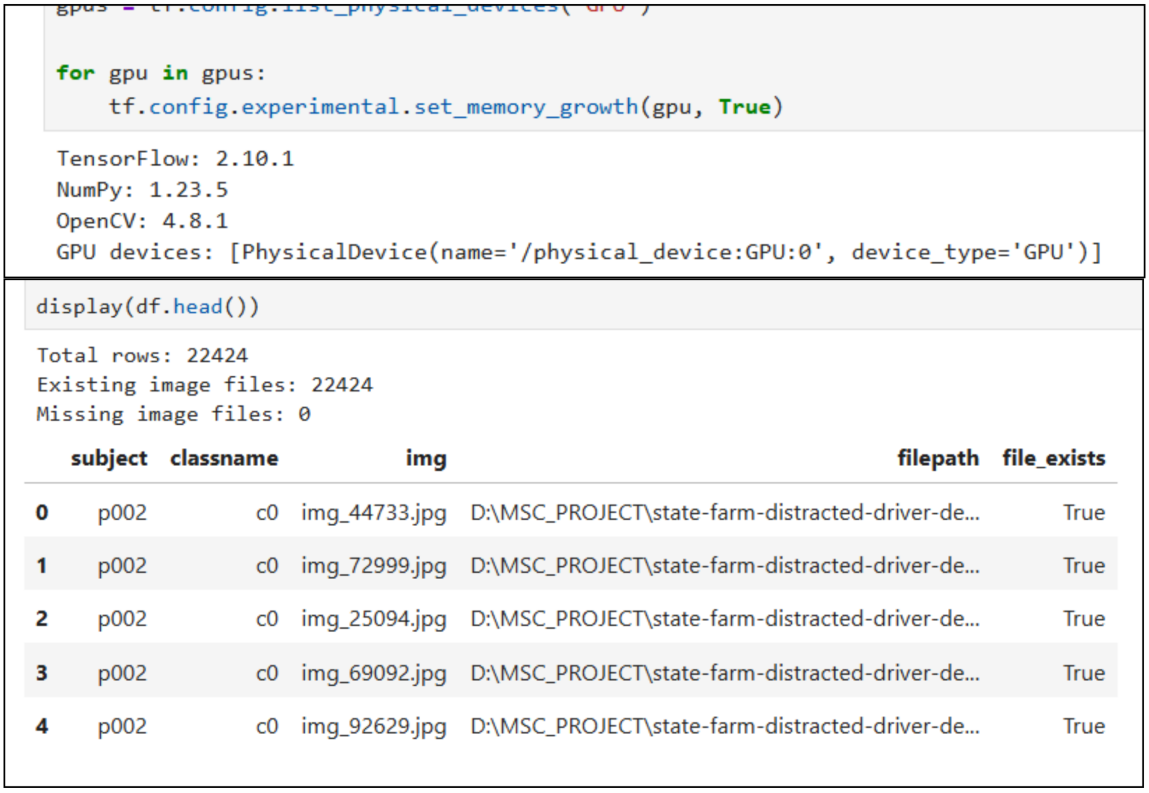
## A Evidence Register

**Table 6:** Project evidence available at the IPR stage.

| Evidence | Location | Purpose |
|---|---|---|
| Dataset exploration | `notebooks/01_dataset_exploration.ipynb` | Image-path checks, environment confirmation, class/subject distributions and sample images. |
| Subject-wise split | `notebooks/02_subject_wise_split.ipynb`; `data/splits/*.csv` | Reproducible partitions and confirmation of zero subject overlap. |
| Baseline CNN | `notebooks/03_baseline_cnn_training.ipynb` | Model structure, checkpoint handling, report and confusion matrix. |
| Transfer learning | `notebooks/04_transfer_learning.ipynb` | MobileNetV2 and EfficientNet-B0 training, fine-tuning and saving issues. |
| Comparative evaluation | `notebooks/05_evaluation_analysis.ipynb` | Comparison tables, latency, FPS and qualitative domain-shift analysis. |
| Survey and planning | `docs/survey/`; `docs/project_plan/` | PRISMA survey, DPP, methodology, risks and timeline. |
| Version control | Private GitHub repository | Notebooks, source, split files, figures, tables and documentation; dataset images and model binaries excluded. |

## B Notebook Execution Evidence

The following screenshots provide supporting evidence from executed project notebooks.



```
for gpu in gpus:
    tf.config.experimental.set_memory_growth(gpu, True)

TensorFlow: 2.10.1
NumPy: 1.23.5
OpenCV: 4.8.1
GPU devices: [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

```
display(df.head())

Total rows: 22424
Existing image files: 22424
Missing image files: 0
```

| | subject | classname | img | filepath | file_exists |
|---|---|---|---|---|---|
| 0 | p002 | c0 | img_44733.jpg | D:\MSC_PROJECT\state-farm-distracted-driver-de... | True |
| 1 | p002 | c0 | img_72999.jpg | D:\MSC_PROJECT\state-farm-distracted-driver-de... | True |
| 2 | p002 | c0 | img_25094.jpg | D:\MSC_PROJECT\state-farm-distracted-driver-de... | True |
| 3 | p002 | c0 | img_69092.jpg | D:\MSC_PROJECT\state-farm-distracted-driver-de... | True |
| 4 | p002 | c0 | img_92629.jpg | D:\MSC_PROJECT\state-farm-distracted-driver-de... | True |

**Figure 8:** Executed notebook output confirming the software environment, GPU detection and dataset-integrity checks.

```
print("Test subjects:", test_df["subject"].nunique())
```

```
Train images: 15899
Validation images: 3439
Test images: 3086
Train subjects: 18
Validation subjects: 4
Test subjects: 4
```

```
print("Success: No subject leakage found.")
```

```
Train-Val subject overlap: set()
Train-Test subject overlap: set()
Val-Test subject overlap: set()
Success: No subject leakage found.
```

**Figure 9:** Executed notebook output confirming partition sizes and the absence of subject overlap.

```
print("MobileNetV2 Test Accuracy:", test_accuracy)
```

```
215/215 [==============================] - 5s 18ms/step - loss: 0.6014 - accuracy: 0.8212
193/193 [==============================] - 3s 17ms/step - loss: 0.4768 - accuracy: 0.8467
MobileNetV2 Validation Loss: 0.6014394760131836
MobileNetV2 Validation Accuracy: 0.8211689591407776
MobileNetV2 Test Loss: 0.4768403172492981
MobileNetV2 Test Accuracy: 0.8467271327972412
```

**Figure 10:** Executed notebook output showing the final MobileNetV2 validation and test evaluation.

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[40], line 3
      1 savedmodel_path = PROJECT_ROOT / "experiments" / "03_efficientnet_b0" / "efficientnetb0_finetuned_savedmodel"
----> 3 best_eff_model.save(savedmodel_path, save_format="tf")
      5 print("Saved EfficientNet-B0 SavedModel to:")
      6 print(savedmodel_path)

File D:\Apps\Anaconda\Anaconda_New\envs\msc_driver_tf_gpu_win\lib\site-packages\keras\utils\traceback_utils.py:70, in filter_traceback.<locals>.e
andler(*args, **kwargs)
     67     filtered_tb = _process_traceback_frames(e.__traceback__)
     68     # To get the full stack trace, call:
     69     # `tf.debugging.disable_traceback_filtering()`
---> 70     raise e.with_traceback(filtered_tb) from None
     71 finally:
     72     del filtered_tb

File D:\Apps\Anaconda\Anaconda_New\envs\msc_driver_tf_gpu_win\lib\json\encoder.py:199, in JSONEncoder.encode(self, o)
    195         return encode_basestring(o)
    196 # This doesn't pass the iterator directly to ''.join() because the
    197 # exceptions aren't as detailed.  The list call should be roughly
    198 # equivalent to the PySequence_Fast that ''.join() would do.
--> 199 chunks = self.iterencode(o, _one_shot=True)
    200 if not isinstance(chunks, (list, tuple)):
    201     chunks = list(chunks)

File D:\Apps\Anaconda\Anaconda_New\envs\msc_driver_tf_gpu_win\lib\json\encoder.py:257, in JSONEncoder.iterencode(self, o, _one_shot)
    252 else:
    253     _iterencode = _make_iterencode(
    254         markers, self.default, _encoder, self.indent, floatstr,
    255         self.key_separator, self.item_separator, self.sort_keys,
    256         self.skipkeys, _one_shot)
--> 257 return _iterencode(o, 0)

TypeError: Unable to serialize [2.0896919 2.1128857 2.1081853] to JSON. Unrecognized type <class 'tensorflow.python.framework.ops.EagerTensor'>.
```

**Figure 11:** TensorFlow/Keras serialisation error encountered during EfficientNet-B0 model saving.

## C Version Control and Supervisor Meeting Evidence

The following evidence shows the project repository structure, version-control activity and the recording of supervisor feedback.

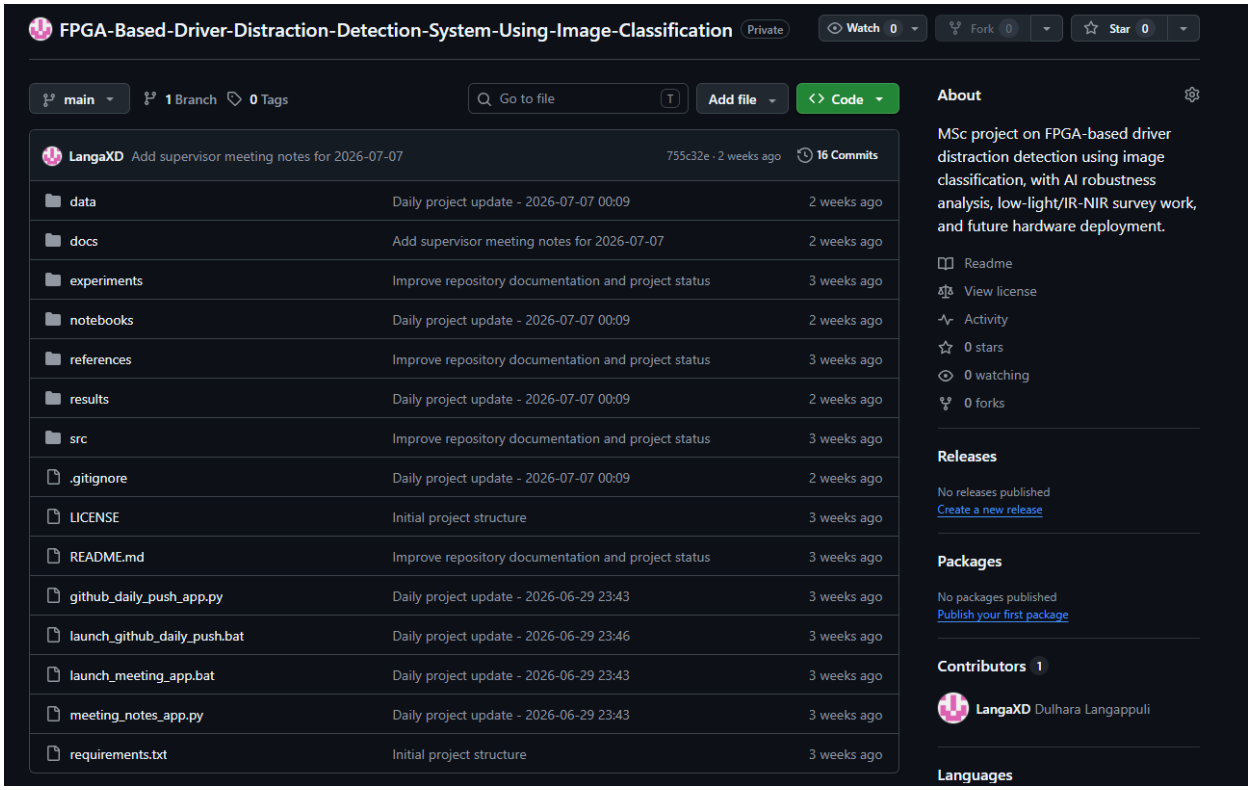



**Figure 12:** Private GitHub repository showing the project structure, development files and commit activity.

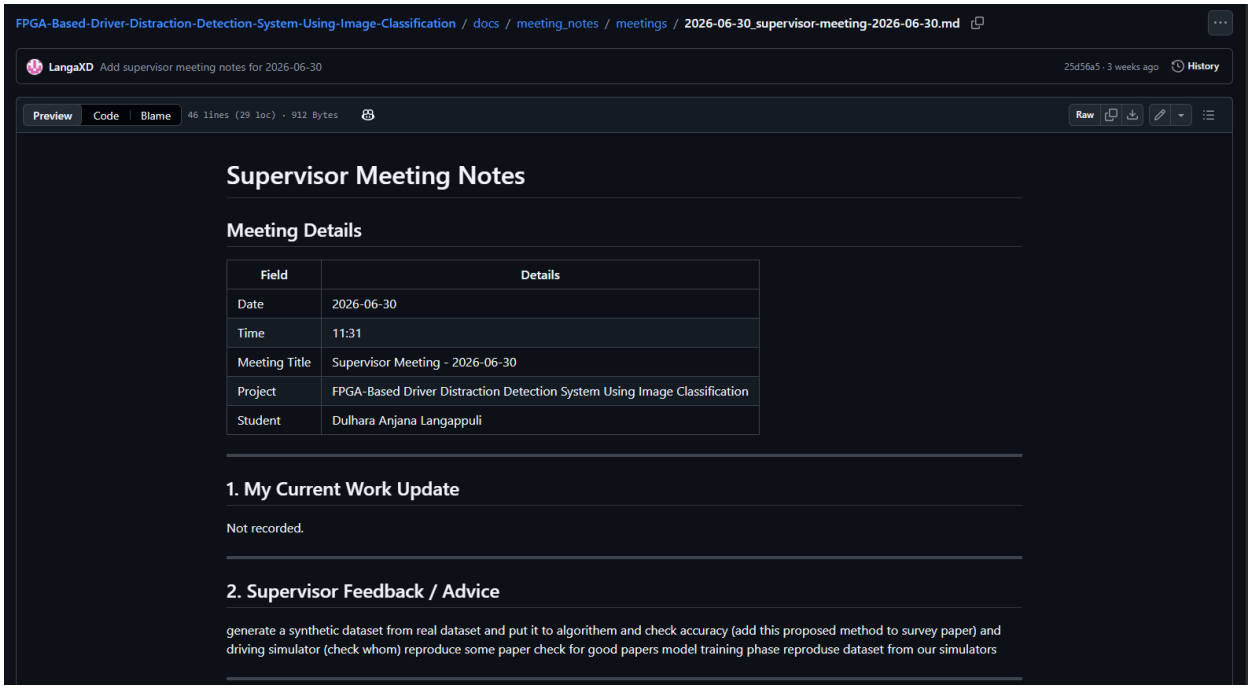



**Figure 13:** Supervisor meeting record dated 30 June 2026 showing the project discussion and feedback received.